

Introduction



Preparation

- 上课前阅读相应的论文
- 回答问题



Labs

- ☐ lab1: MapReduce
- ☐ lab2: Raft
- ☐ lab3: KVDataBase (Raft)
- ☐ lab4: Sharded KV Service



Source

课程表/论文/实验要求

- ☐ <https://pdos.csail.mit.edu/6.824/schedule.html>

实验代码框架/测试

- ☐ [git clone git://g.csail.mit.edu/6.824-golabs-2022](https://g.csail.mit.edu/6.824-golabs-2022)

课程视频

- ☐ https://www.bilibili.com/video/BV1R7411t71W/?vd_source=7b40dccd7c6e749fed413a4dde67b693



代码框架

1. 核心代码

这些文件夹对应了分布式系统的三个阶段：**离线计算 -> 共识协议 -> 应用服务。**

- **mr/ (MapReduce): Lab 1** 的所在地。你需要在这里实现分布式 MapReduce，处理 Master (Coordinator) 和 Worker 之间的任务分发。

- **raft/ (Raft)**: Lab 2 的核心。这是整个课程的灵魂，你需要在这里手写 Raft 算法（选举、日志同步、持久化、快照）。
- **kvraft/ (KV Raft)**: Lab 3。基于你自己写的 Raft 协议，构建一个支持强一致性的分布式 Key-Value 数据库。
- **shardctrler/ & shardkv/**: Lab 4（最难的部分）。实现分片（Sharding）控制器和分片数据库。通过将数据切片，实现系统的水平扩展。

2. 底层设施

这些是支撑实验运行的“脚手架”：

- **labrpc/**：模拟网络层。它模拟了不稳定的网络环境（丢包、延迟、重传）。你会在这里调用 `Call()` 函数发送远程过程调用。
- **labgob/**：序列化工具。基于 Go 原生的 `encoding/gob` 进行了封装，用于在 RPC 发送和 Persist 持久化时转换数据格式。如果报错“未注册类型”，通常和这里有关。
- **models/**：定义了一些通用的接口和模型。

3. 测试工具

- **main/**：包含了一些可执行程序的入口。比如 MapReduce 的测试脚本、以及启动单个 Raft 节点的示例。
- **mrapps/**：提供了一些简单的 MapReduce 应用程序（比如单词计数 WordCount、文本倒排索引），用来测试你的 `mr/` 实现是否正确。
- **porcupine/**：线性一致性检查工具。这是一个非常强大的工具，它会根据你运行的日志，自动画图并分析你的分布式系统是否真正满足“强一致性”。

实验内容

MIT 6.824（现课程编号为 **6.5840**）是分布式系统领域的“金标准”课程。实验（Labs）的含金量极高，主要围绕着**从零实现一个基于 Raft 的强一致性分布式键值存储系统**展开。

以下是四个核心实验的内容及其对应的必读论文：

Lab 1: MapReduce

实验内容：

实现一个分布式的 MapReduce 框架，模仿 Google 著名的 MapReduce 系统。

- 你需要编写一个 **Coordinator（协调者）** 和多个 **Worker（工作者）**。
- Worker 通过 RPC 向 Coordinator 请求任务（Map 或 Reduce），读取文件并处理。

- **挑战点：** 处理 Worker 崩溃或运行缓慢（Stragglers）的情况，确保任务最终能正确完成。

必读论文：

- 《MapReduce: Simplified Data Processing on Large Clusters》（Jeff Dean and Sanjay Ghemawat）

Lab 2: Raft (最难点)

这是整个课程的核心，通常分为 A, B, C, D 四个子部分。

实验内容：

实现 **Raft 一致性算法**，让一组状态机在存在网络分区或服务器宕机时达成共识。

- **2A (Leader Election):** 实现心跳机制和领导者选举。
- **2B (Log Replication):** 实现日志同步，确保所有节点日志顺序一致。
- **2C (Persistence):** 实现状态持久化，以便节点重启后能恢复数据。
- **2D (Log Compaction):** 实现日志压缩（Snapshot），防止日志无限增长。

必读论文：

- 《In Search of an Understandable Consensus Algorithm (Extended Version)》（Diego Ongaro and John Ousterhout）

Lab 3: Fault-tolerant Key/Value Service

实验内容：

在 Lab 2 的 Raft 基础上，构建一个可以容错的 **分布式键值数据库**。

- 支持 `Put`、`Append` 和 `Get` 操作。
- **挑战点：** 确保 **线性一致性（Linearizability）**。即使客户端请求重发、Leader 切换，系统也要保证每个操作只执行一次（幂等性）。

必读论文：

- **不需要新论文**，但需要深入理解 Raft 论文中的 **第 7 节（Client Interaction）**。

Lab 4: Sharded Key/Value Service

实验内容：

实现一个 **分片（Sharded）** 的键值存储系统，通过增加机器来扩展系统容量（水平扩展）。

- 你需要实现两个部分：**ShardController**（负责管理哪些分片在哪些服务器组上）和 **ShardServer**（负责实际存储数据）。

- **挑战点：** 在不同服务器组之间迁移分片（Shard Migration），同时还要保证数据的一致性和可用性。

必读论文：

- **不需要特定论文**，但通常会参考设计思路类似于 **Google Spanner** 或 **BigTable** 的分片管理机制。

实验代码

在 MIT 6.5840 (6.824) 的实验中，代码主要组织在 `src` 目录下。虽然文件很多，但你真正需要修改和关注的文件是比较固定的。

以下是每个实验涉及的核心文件清单：

Lab 1: MapReduce

这个实验的代码主要在 `src/mr/` 目录下。

- `mr/coordinator.go` (核心修改): 负责任务分配、状态监控和容错逻辑。
- `mr/worker.go` (核心修改): 负责读取输入、执行用户函数、将中间结果写入文件，并与 Coordinator 通信。
- `mr/rpc.go` (核心修改): 定义 Coordinator 和 Worker 之间交互的 RPC 结构体 (Args 和 Reply)。
- `main/mrcoordinator.go`: 启动入口，你通常不需要改。
- `main/mrworker.go`: 启动入口，通常不需要改。
- `main/test-mr.sh`: **非常重要**，这是官方提供的集成测试脚本。

Lab 2: Raft (最核心部分)

代码集中在 `src/raft/` 目录下。

- `raft/raft.go` (最核心修改): 所有的 Raft 逻辑（选举、日志同步、持久化、快照）都在这个文件里。你需要定义 `Raft` 结构体，实现 `RequestVote` 和 `AppendEntries` 等 RPC 处理函数。
- `raft/util.go`: 包含调试日志函数 `DPrintf`。建议在这里自定义你的日志逻辑。
- `raft/persister.go`: 提供持久化接口，用于 2C 实验中保存硬状态 (CurrentTerm, VotedFor, Log)。
- `raft/test_test.go`: 官方测试用例。当你遇到 Bug 时，看这里的测试逻辑能帮你定位问题。

Lab 3: Fault-tolerant KV Service

代码集中在 `src/kvraft/` 目录下，它位于 Raft 层之上。

- `kvraft/server.go` (核心修改): 实现 KV 服务器逻辑。它会调用 Raft 层来达成一致，并管理状态机。
- `kvraft/client.go` (核心修改): 实现客户端逻辑。需要处理 RPC 超时、寻找当前 Leader 以及确保操作的幂等性。
- `kvraft/common.go`: 定义 KV 服务的 RPC 接口 (Put/Append/Get)。

Lab 4: Sharded KV Service

代码集中在 `src/shardctrler/` 和 `src/shardkv/` 目录下。

- `shardctrler/server.go` & `client.go`: 实现分片控制器。负责决定哪些数据分片 (Shards) 由哪些服务器组 (Replica Groups) 负责。
- `shardkv/server.go` & `client.go`: 实现具体存储数据的服务器。最难的点在于处理分片在不同组之间的迁移 (Migration)。
- `shardkv/common.go`: 定义分片迁移和数据读写的 RPC。

关键文件建议

1. `src/raft/raft.go` 是你投入时间最多的地方，建议在开始前对着 **Raft 论文中的 Figure 2** 把结构体和接口先搭好。
2. `main/pg-*.txt`: 在 Lab 1 中，这是输入数据（电子书内容），如果你遇到 Plugin 报错，通常是因为 Go 插件编译环境的问题，可以检查 `Makefile` 或手动使用 `go build -buildmode=plugin`。
3. **日志文件**: 分布式实验很难断点调试，养成将输出重定向到文件的习惯：`go test -run 2A > out.log`。